

Design for class "PaymentService"

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	orderRepository	OrderRepository		Repository để truy xuất thông tin đơn hàng
2	paymentTransactionRepository	PaymentTransactionRepository		Repository để lưu trữ lịch sử giao dịch
3	paymentStrategyRegistry	PaymentStrategyRegistry		Registry để lấy chiến lược thanh toán phù hợp (Strategy Pattern)
4	emailService	EmailService		Service gửi email xác nhận đơn hàng
5	logger	Logger	new Logger()	Ghi log hệ thống

Table 2. Operation design

#	Name	Return type	Description/Purpose
1	convertVNDToUSD()	number	Chuyển đổi tiền tệ từ VND sang USD (Private)
2	convertUSDToVND()	number	Chuyển đổi tiền tệ từ USD sang VND (Private)
3	createPayment()	Promise<PaymentResult>	Khởi tạo giao dịch thanh toán mới dựa trên phương thức được chọn
4	verifyPayment()	Promise<PaymentResult>	Xác thực kết quả thanh toán từ cổng thanh toán và cập nhật trạng thái đơn hàng
5	forceSuccessPayment()	Promise<void>	Cưỡng ép cập nhật trạng thái đơn hàng thành công (Dùng cho Demo/Test)

Method design

1. createPayment

- **Parameter:** createPaymentDto: CreatePaymentDto
- **Exception:** Error (Invalid order amount)
- **Method:**
 1. Lấy chiến lược thanh toán (Strategy) từ paymentStrategyRegistry dựa trên createPaymentDto.method.
 2. Lấy tổng tiền đơn hàng từ orderRepository.
 3. Kiểm tra tính hợp lệ của số tiền (lớn hơn 0).
 4. Gọi phương thức convertCurrency của strategy để chuyển đổi tiền tệ.
 5. Tạo đối tượng context PaymentContext.
 6. Gọi strategy.createPayment(paymentContext) để khởi tạo giao dịch với bên thứ 3.
 7. Trích xuất URL thanh toán từ kết quả trả về (nếu có).
 8. Lưu thông tin giao dịch vào database với trạng thái PENDING thông qua paymentTransactionRepository.
 9. Trả về kết quả giao dịch PaymentResult.

2. verifyPayment

- **Parameter:** token: string, orderId: string, method: PAYMENT_METHOD
- **Exception:** None
- **Method:**

1. Lấy chiến lược thanh toán tương ứng từ Registry.
2. Gọi `strategy.capturePayment(token)` để xác thực giao dịch với cổng thanh toán.
3. Cập nhật trạng thái giao dịch trong database (SUCCESS hoặc FAILED).
4. Nếu thanh toán thành công:
 - (a) Cập nhật trạng thái đơn hàng thành `ORDER_STATUS.PROCESSING`.
 - (b) Tìm thông tin đơn hàng và khách hàng.
 - (c) Gọi `emailService.sendOrderConfirmation` để gửi email xác nhận.
5. Trả về kết quả xác thực.

3. forceSuccessPayment

- **Parameter:** orderId: string

- **Exception:** None

- **Method:**

1. Cập nhật trực tiếp trạng thái đơn hàng thành `ORDER_STATUS.PROCESSING`.
2. Tìm thông tin khách hàng và gửi email xác nhận (tương tự như `verifyPayment`).
3. Ghi log hành động demo.